A Monero data model and API specification

implemented by monero-csharp, for .NET
using monero v0.18.5.4 'Fluorine Fermi'

Last updated November 1, 2026



C# implementation: https://github.com/libmonero/monero-csharp

C ABI used by the wallet and the daemon: https://github.com/libmonero/monero-c

Document source: https://github.com/libmonero/monero-csharp/blob/main/docs/monero-spec.drawio

Document created and revisable using: https://www.draw.io/

How to read the C# tables
The rows are the public members of MoneroDaemon and MoneroWallet, and of the model classes, in C# types. A method that talks to the node or to the wallet is async and returns Task<T>, and takes a CancellationToken ct = default as its last parameter, which the rows leave out. A property with a ? is nullable, and null means that the value is not known.

Monero Daemon Interface 1/3 - MoneroDaemon

AddListener(MoneroDaemonListener listener): void

RemoveListener(MoneroDaemonListener listener): void

GetListeners(): List<MoneroDaemonListener>

RemoveListeners(): void

GetVersionAsync(): Task<MoneroVersion>

IsTrustedAsync(): Task<bool>

GetHeightAsync(): Task<ulong>

GetInfoAsync(): Task<MoneroDaemonInfo>

GetSyncInfoAsync(): Task<MoneroDaemonSyncInfo>

GetNetworkStatsAsync(): Task<MoneroDaemonNetworkStats>

GetHardForkInfoAsync(): Task<MoneroHardForkInfo>

GetAltChainsAsync(): Task<List<MoneroAltChain>>

GetAltBlockHashesAsync(): Task<List<string>>

WaitForNextBlockHeaderAsync(): Task<MoneroBlockHeader>

GetBlockHashAsync(ulong height): Task<string>

GetLastBlockHeaderAsync(): Task<MoneroBlockHeader>

GetBlockHeaderByHashAsync(string hash): Task<MoneroBlockHeader>

GetBlockHeaderByHeightAsync(ulong height): Task<MoneroBlockHeader>

GetBlockHeadersByRangeAsync(ulong startHeight, ulong endHeight): Task<List<MoneroBlockHeader>>

GetBlockByHashAsync(string hash): Task<MoneroBlock>

GetBlockByHeightAsync(ulong height): Task<MoneroBlock>

GetBlocksByHashAsync(IReadOnlyList<string> blockHashes, ulong startHeight, bool prune,
ulong maxBlockCount=0): Task<MoneroGetBlocksByHashResult>

GetBlocksByHeightAsync(IReadOnlyList<ulong> heights): Task<List<MoneroBlock>>

GetBlocksByRangeAsync(ulong? startHeight, ulong? endHeight): Task<List<MoneroBlock>>

GetBlocksByRangeChunkedAsync(ulong? startHeight, ulong? endHeight,
ulong? maxChunkSize=null): Task<List<MoneroBlock>>

GetBlockHashesAsync(IReadOnlyList<string> blockHashes): Task<MoneroGetBlockHashesResult>

GetTxAsync(string txHash, bool prune=false): Task<MoneroTx?>

GetTxsAsync(IReadOnlyList<string> txHashes, bool prune=false): Task<List<MoneroTx>>

GetTxHexAsync(string txHash, bool prune=false): Task<string?>

GetTxHexesAsync(IReadOnlyList<string> txHashes, bool prune=false): Task<List<string>>

SubmitTxHexAsync(string txHex, bool doNotRelay=false): Task<MoneroSubmitTxResult>

RelayTxByHashAsync(string txHash): Task

RelayTxsByHashAsync(IReadOnlyList<string> txHashes): Task

Monero Daemon Interface 2/3 - MoneroDaemon

GetTxPoolAsync(): Task<List<MoneroTx>>

GetTxPoolHashesAsync(): Task<List<string>>

GetTxPoolBacklogAsync(): Task<List<MoneroTxBacklogEntry>>

GetTxPoolStatsAsync(): Task<MoneroTxPoolStats>

FlushTxPoolAsync(): Task

FlushTxPoolAsync(string hash): Task

FlushTxPoolAsync(IReadOnlyList<string> hashes): Task

GetKeyImageSpentStatusAsync(string keyImage): Task<MoneroKeyImageSpentStatus>

GetKeyImageSpentStatusesAsync(IReadOnlyList<string> keyImages): Task<List<MoneroKeyImageSpentStatus>>

GetOutputIndicesAsync(string txHash): Task<List<ulong>>

GetOutputsAsync(IReadOnlyList<MoneroOutput> outputs): Task<List<MoneroOutput>>

GetOutputHistogramAsync(IReadOnlyList<ulong> amounts, int? minCount=null, int? maxCount=null, bool? isUnlocked=null,
    int? recentCutoff=null): Task<List<MoneroOutputHistogramEntry>>

GetOutputDistributionAsync(IReadOnlyList<ulong> amounts, bool? isCumulative=null, ulong? startHeight=null,
    ulong? endHeight=null): Task<List<MoneroOutputDistributionEntry>>

GetFeeEstimateAsync(ulong graceBlocks=0): Task<MoneroFeeEstimate>

GetBlockTemplateAsync(string walletAddress, int? reserveSize=null): Task<MoneroBlockTemplate>

GetMinerDataAsync(): Task<MoneroMinerData>

CalculatePowAsync(uint majorVersion, ulong height, string blockBlob, string seedHash): Task<string>

AddAuxiliaryPowAsync(string blockTemplateBlob,
    IReadOnlyList<MoneroAuxiliaryPow> auxPow): Task<MoneroAddAuxiliaryPowResult>

GetMinerTxSumAsync(ulong height, ulong numBlocks): Task<MoneroMinerTxSum>

StartMiningAsync(string address, ulong? numThreads=null, bool? isBackground=null, bool? ignoreBattery=null): Task

StopMiningAsync(): Task

GetMiningStatusAsync(): Task<MoneroMiningStatus>

GenerateBlocksAsync(string walletAddress, ulong numBlocks, string? prevBlockHash=null,
    uint? startingNonce=null): Task<MoneroGenerateBlocksResult>

SubmitBlockAsync(string blockBlob): Task

SubmitBlocksAsync(IReadOnlyList<string> blockBlobs): Task

GetPeersAsync(): Task<List<MoneroPeer>>

GetKnownPeersAsync(): Task<List<MoneroPeer>>

GetPublicPeersAsync(bool includeOffline=false): Task<List<MoneroPeer>>

SetOutgoingPeerLimitAsync(int limit): Task

SetIncomingPeerLimitAsync(int limit): Task

GetPeerBansAsync(): Task<List<MoneroBan>>

GetPeerBanAsync(string address): Task<MoneroBan>

SetPeerBansAsync(IReadOnlyList<MoneroBan> bans): Task

SetPeerBanAsync(MoneroBan ban): Task

GetDownloadLimitAsync(): Task<int>

Monero Daemon Interface 3/3 - MoneroDaemon

SetDownloadLimitAsync(int limit): Task<int>

ResetDownloadLimitAsync(): Task<int>

GetUploadLimitAsync(): Task<int>

SetUploadLimitAsync(int limit): Task<int>

ResetUploadLimitAsync(): Task<int>

PruneBlockchainAsync(bool check): Task<MoneroPruneResult>

SaveBlockchainAsync(): Task

PopBlocksAsync(ulong numBlocks): Task<ulong>

FlushCacheAsync(bool badBlocks=false): Task

SetBootstrapDaemonAsync(string address, string username="", string password="", string proxy=""): Task

RemoveBootstrapDaemonAsync(): Task

SetLogHashRateAsync(bool isVisible): Task

SetLogLevelAsync(int level): Task

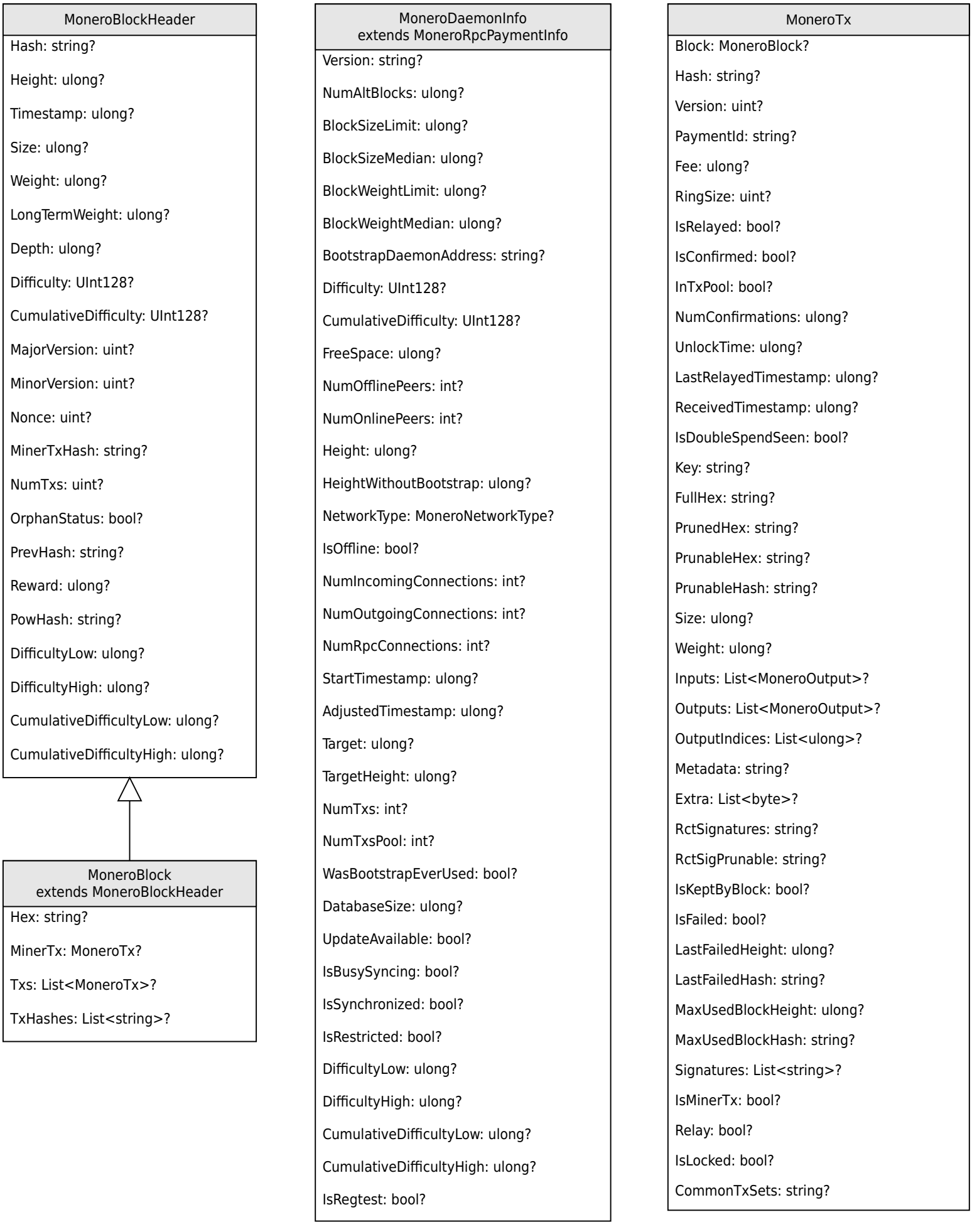
SetLogCategoriesAsync(string categories=""): Task<string>

CheckForUpdateAsync(): Task<MoneroDaemonUpdateCheckResult>

DownloadUpdateAsync(string path=""): Task<MoneroDaemonUpdateDownloadResult>

StopAsync(): Task

Monero Daemon Types 1/3



Monero Daemon Types 1/3 (continued)

MoneroOutput
Tx: MoneroTx?
KeyImage: MoneroKeyImage?
Amount: ulong?
Index: ulong?
RingOutputIndices: List<ulong>?
StealthPublicKey: string?
Mask: string?

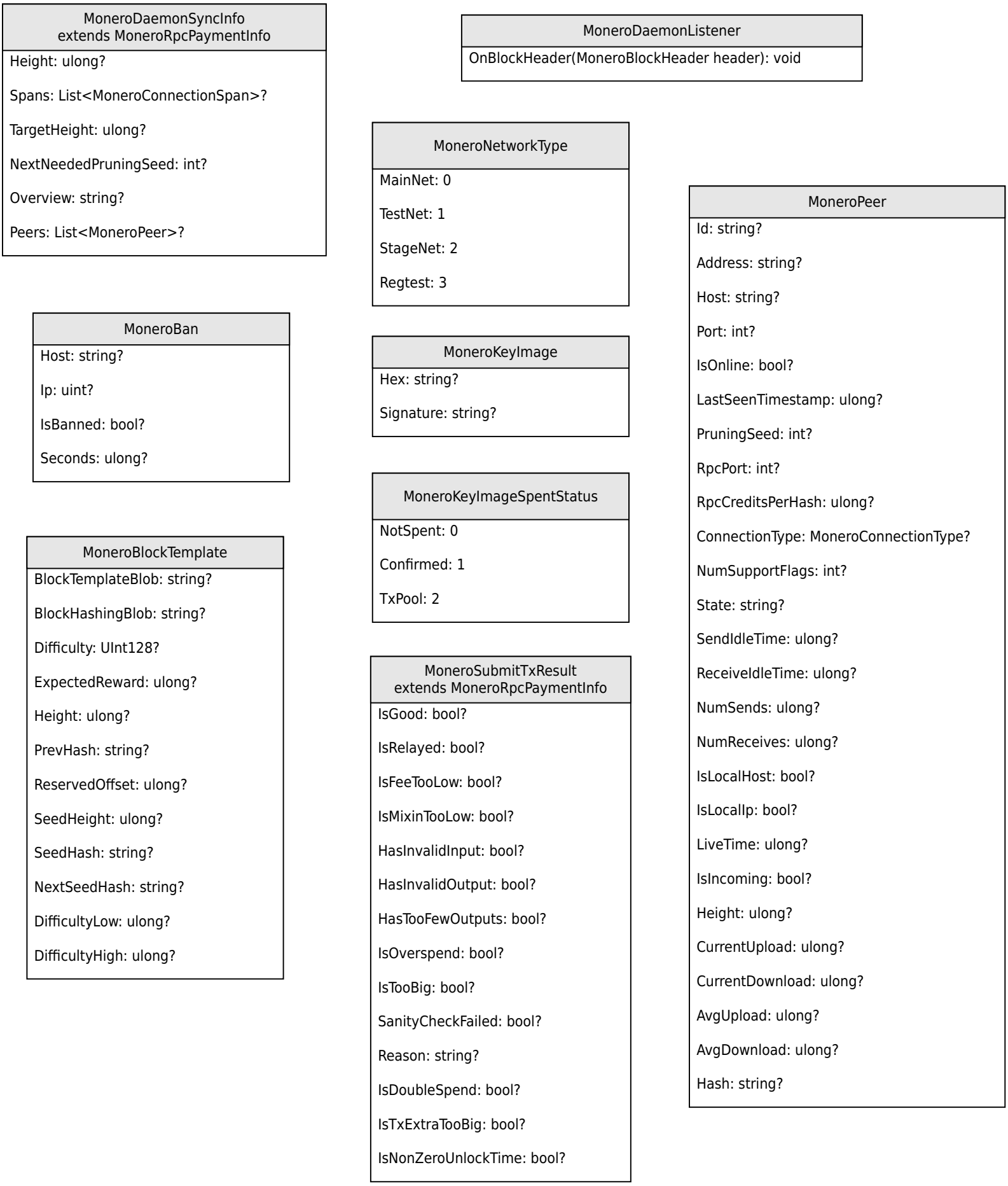
MoneroBlocksResult
Blocks: List<MoneroBlock>?

MoneroGetBlockHashesResult
StartHeight: ulong?
CurrentHeight: ulong?
Hashes: List<string>?

MoneroGetBlocksByHashResult
CurrentHeight: ulong?
Blocks: List<MoneroBlock>?

MoneroRpcPaymentInfo
Credits: ulong?
TopBlockHash: string?

Monero Daemon Types 2/3



Monero Daemon Types 2/3 (continued)

MoneroConnectionSpan
ConnectionId: string?
NumBlocks: ulong?
RemoteAddress: string?
Rate: ulong?
Speed: ulong?
Size: ulong?
StartHeight: ulong?

MoneroAddAuxiliaryPowResult extends MoneroRpcPaymentInfo
BlockTemplateBlob: string?
BlockHashingBlob: string?
MerkleRoot: string?
MerkleTreeDepth: uint?
AuxPow: List<MoneroAuxiliaryPow>?

MoneroAuxiliaryPow
Id: string?
Hash: string?

MoneroGenerateBlocksResult
Height: ulong?
BlockHashes: List<string>?

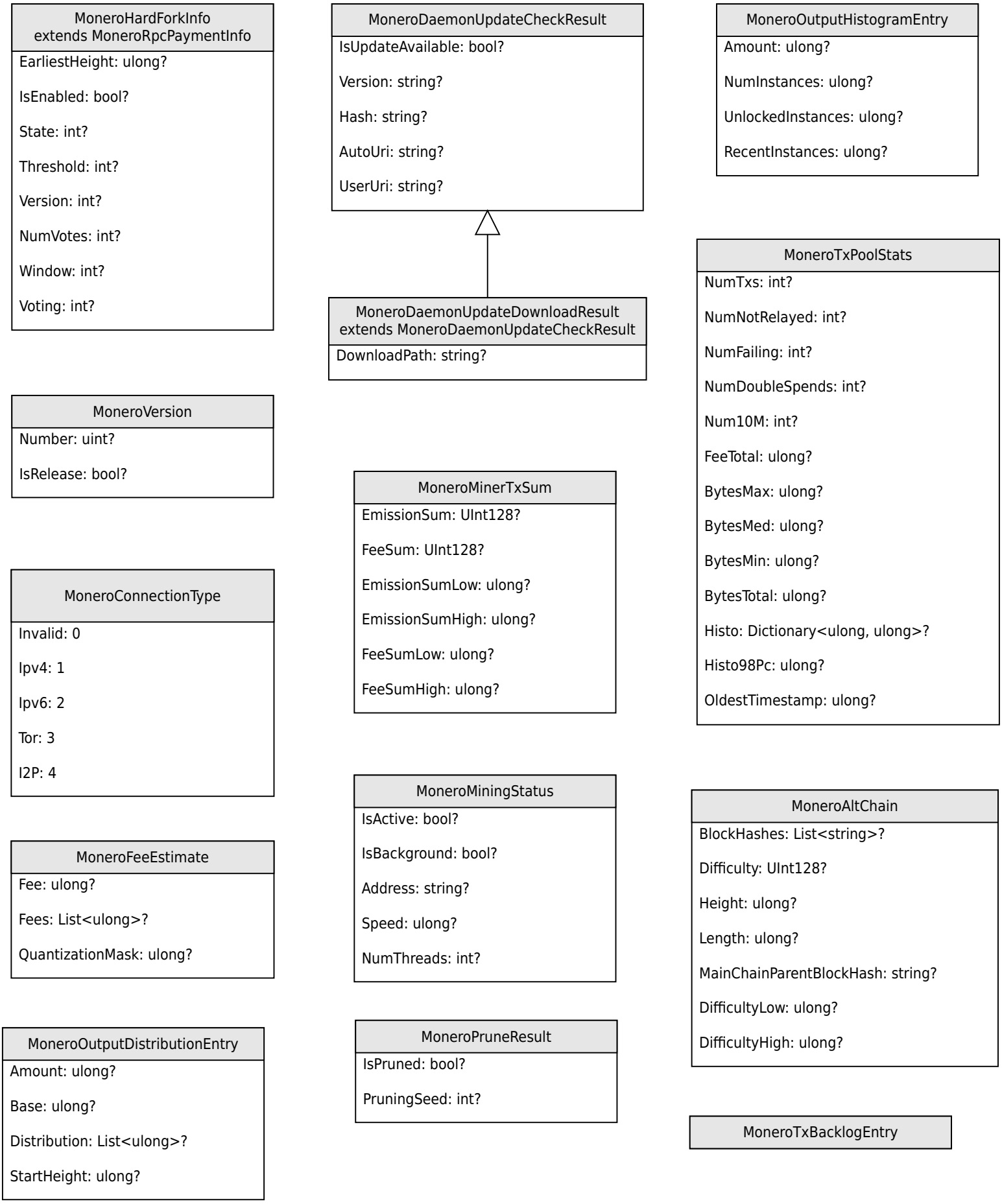
MoneroMinerData extends MoneroRpcPaymentInfo
MajorVersion: uint?
Height: ulong?
PrevHash: string?
SeedHash: string?
Difficulty: string?
MedianWeight: ulong?
AlreadyGeneratedCoins: ulong?
TxPoolBacklog: List<MoneroTx>?

Monero Daemon Types 2/3 (continued)

MoneroRpcConnection
Uri: string?
Username: string?
Password: string?
ProxyUri: string?
ZmqUri: string?
Priority: int?
TimeoutMs: uint?
ResponseTime: ulong?
SslVerify: bool?
IsOnline: bool?
IsAuthenticated: bool?

MoneroDaemonNetworkStats extends MoneroRpcPaymentInfo
StartTime: ulong?
TotalPacketsIn: ulong?
TotalBytesIn: ulong?
TotalPacketsOut: ulong?
TotalBytesOut: ulong?

Monero Daemon Types 3/3



Monero Wallet Interface 1/4 - MoneroWallet

AddListener(MoneroWalletListener listener): void

RemoveListener(MoneroWalletListener listener): void

GetListeners(): List<MoneroWalletListener>

RemoveListeners(): void

GetVersionAsync(): Task<MoneroVersion>

GetPathAsync(): Task<string>

GetNetworkTypeAsync(): Task<MoneroNetworkType>

IsViewOnlyAsync(): Task<bool>

IsClosedAsync(): Task<bool>

GetSeedAsync(): Task<string>

GetSeedLanguageAsync(): Task<string>

GetPrivateViewKeyAsync(): Task<string>

GetPrivateSpendKeyAsync(): Task<string>

GetPublicViewKeyAsync(): Task<string>

GetPublicSpendKeyAsync(): Task<string>

GetPrimaryAddressAsync(): Task<string>

GetAddressAsync(uint accountIdx, uint subaddressIdx): Task<string>

GetAddressIndexAsync(string address): Task<MoneroSubaddress>

GetIntegratedAddressAsync(string? standardAddress=null, string? paymentId=null): Task<MoneroIntegratedAddress>

DecodeIntegratedAddressAsync(string integratedAddress): Task<MoneroIntegratedAddress>

GetAttributeAsync(string key): Task<string?>

SetAttributeAsync(string key, string value): Task

GetDefaultFeePriorityAsync(): Task<MoneroTxPriority>

SaveAsync(): Task

MoveToAsync(string path, string password): Task

ChangePasswordAsync(string oldPassword, string newPassword): Task

CloseAsync(bool save=false): Task

GetDaemonConnectionAsync(): Task<MoneroRpcConnection?>

SetDaemonConnectionAsync(MoneroRpcConnection? connection, bool? isTrusted=null): Task

SetDaemonConnectionAsync(string uri, string? username=null, string? password=null, string? proxyUri=null,
bool? isTrusted=null): Task

IsConnectedToDaemonAsync(): Task<bool>

IsDaemonSyncedAsync(): Task<bool>

IsDaemonTrustedAsync(): Task<bool>

IsSyncedAsync(): Task<bool>

GetHeightAsync(): Task<ulong>

GetDaemonHeightAsync(): Task<ulong>

Monero Wallet Interface 2/4 - MoneroWallet

GetDaemonMaxPeerHeightAsync(): Task<ulong>

GetHeightByDateAsync(int year, int month, int day): Task<ulong>

GetRestoreHeightAsync(): Task<ulong>

SetRestoreHeightAsync(ulong restoreHeight): Task

SyncAsync(ulong? startHeight=null, MoneroWalletListener? listener=null): Task<MoneroSyncResult>

StartSyncingAsync(ulong syncPeriodInMs=10000): Task

StopSyncingAsync(): Task

WaitForNextBlockAsync(): Task<ulong>

ScanTxsAsync(IReadOnlyList<string> txHashes): Task

RescanSpentAsync(): Task

RescanBlockchainAsync(): Task

StartMiningAsync(ulong? numThreads=null, bool? backgroundMining=null, bool? ignoreBattery=null): Task

StopMiningAsync(): Task

GetBalanceAsync(uint? accountIdx=null, uint? subaddressIdx=null): Task<ulong>

GetUnlockedBalanceAsync(uint? accountIdx=null, uint? subaddressIdx=null): Task<ulong>

GetAccountsAsync(bool includeSubaddresses=false, string? tag=null): Task<List<MoneroAccount>>

GetAccountAsync(uint accountIdx, bool includeSubaddresses=false): Task<MoneroAccount>

CreateAccountAsync(string? label=null): Task<MoneroAccount>

GetAccountTagsAsync(): Task<List<MoneroAccountTag>>

TagAccountsAsync(string tag, IReadOnlyList<uint> accountIndices): Task

UntagAccountsAsync(IReadOnlyList<uint> accountIndices): Task

SetAccountTagLabelAsync(string tag, string label): Task

GetSubaddressesAsync(uint accountIdx, IReadOnlyList<uint>? subaddressIndices=null): Task<List<MoneroSubaddress>>

GetSubaddressAsync(uint accountIdx, uint subaddressIdx): Task<MoneroSubaddress>

CreateSubaddressAsync(uint accountIdx, string? label=null): Task<MoneroSubaddress>

SetSubaddressLabelAsync(uint accountIdx, uint subaddressIdx, string? label=null): Task

GetTxsAsync(MoneroTxQuery? query=null): Task<List<MoneroTxWallet>>

GetTxsAsync(IReadOnlyList<string> txHashes): Task<List<MoneroTxWallet>>

GetTxAsync(string txHash): Task<MoneroTxWallet?>

GetTransfersAsync(MoneroTransferQuery? query=null): Task<List<MoneroTransfer>>

GetTransfersAsync(uint accountIdx, uint? subaddressIdx=null): Task<List<MoneroTransfer>>

GetOutputsAsync(MoneroOutputQuery? query=null): Task<List<MoneroOutputWallet>>

ExportOutputsAsync(bool all=false): Task<string>

ImportOutputsAsync(string outputsHex): Task<int>

ExportKeyImagesAsync(bool all=false): Task<MoneroKeyImageExportResult>

ImportKeyImagesAsync(IReadOnlyList<MoneroKeyImage> keyImages, ulong offset=0): Task<MoneroKeyImageImportResult>

FreezeOutputAsync(string keyImage): Task

Monero Wallet Interface 3/4 - MoneroWallet

ThawOutputAsync(string keyImage): Task

IsOutputFrozenAsync(string keyImage): Task<bool>

CreateTxAsync(MoneroTxConfig config): Task<MoneroTxWallet>

CreateTxsAsync(MoneroTxConfig config): Task<List<MoneroTxWallet>>

SweepOutputAsync(MoneroTxConfig config): Task<MoneroTxWallet>

SweepUnlockedAsync(MoneroTxConfig config): Task<List<MoneroTxWallet>>

SweepDustAsync(bool relay=false): Task<List<MoneroTxWallet>>

RelayTxAsync(string txMetadata): Task<string>

RelayTxAsync(MoneroTxWallet tx): Task<string>

RelayTxsAsync(IReadOnlyList<string> txMetadatas): Task<List<string>>

RelayTxsAsync(IReadOnlyList<MoneroTxWallet> txs): Task<List<string>>

DescribeTxSetAsync(MoneroTxSet txSet): Task<MoneroTxSet>

DescribeUnsignedTxSetAsync(string unsignedTxHex): Task<MoneroTxSet>

DescribeMultisigTxSetAsync(string multisigTxHex): Task<MoneroTxSet>

SignTxsAsync(string unsignedTxHex): Task<MoneroTxSet>

SubmitTxsAsync(string signedTxHex): Task<List<string>>

GetTxNoteAsync(string txHash): Task<string>

GetTxNotesAsync(IReadOnlyList<string> txHashes): Task<List<string>>

SetTxNoteAsync(string txHash, string note): Task

SetTxNotesAsync(IReadOnlyList<string> txHashes, IReadOnlyList<string> notes): Task

GetAddressBookEntriesAsync(IReadOnlyList<ulong>? indices=null): Task<List<MoneroAddressBookEntry>>

AddAddressBookEntryAsync(string address, string? description=null): Task<ulong>

EditAddressBookEntryAsync(ulong index, string? address, string? description): Task

DeleteAddressBookEntryAsync(ulong index): Task

GetPaymentUriAsync(MoneroTxConfig config): Task<string>

ParsePaymentUriAsync(string uri): Task<MoneroTxConfig>

SignMessageAsync(string message,
    MoneroMessageSignatureType signatureType=MoneroMessageSignatureType.SignWithSpendKey, uint accountIdx=0,
    uint subaddressIdx=0): Task<string>

VerifyMessageAsync(string message, string address, string signature): Task<MoneroMessageSignatureResult>

GetTxKeyAsync(string txHash): Task<string>

CheckTxKeyAsync(string txHash, string txKey, string address): Task<MoneroCheckTx>

GetTxProofAsync(string txHash, string address, string? message=null): Task<string>

CheckTxProofAsync(string txHash, string address, string? message, string signature): Task<MoneroCheckTx>

GetSpendProofAsync(string txHash, string? message=null): Task<string>

CheckSpendProofAsync(string txHash, string? message, string signature): Task<bool>

GetReserveProofWalletAsync(string? message=null): Task<string>

GetReserveProofAccountAsync(uint accountIdx, ulong amount, string? message=null): Task<string>

Monero Wallet Interface 4/4 - MoneroWallet

CheckReserveProofAsync(string address, string? message, string signature): Task<MoneroCheckReserve>

IsMultisigAsync(): Task<bool>

GetMultisigInfoAsync(): Task<MoneroMultisigInfo>

IsMultisigImportNeededAsync(): Task<bool>

PrepareMultisigAsync(): Task<string>

MakeMultisigAsync(IReadOnlyList<string> multisigHexes, int threshold, string password): Task<string>

ExchangeMultisigKeysAsync(IReadOnlyList<string> multisigHexes, string password): Task<MoneroMultisigInitResult>

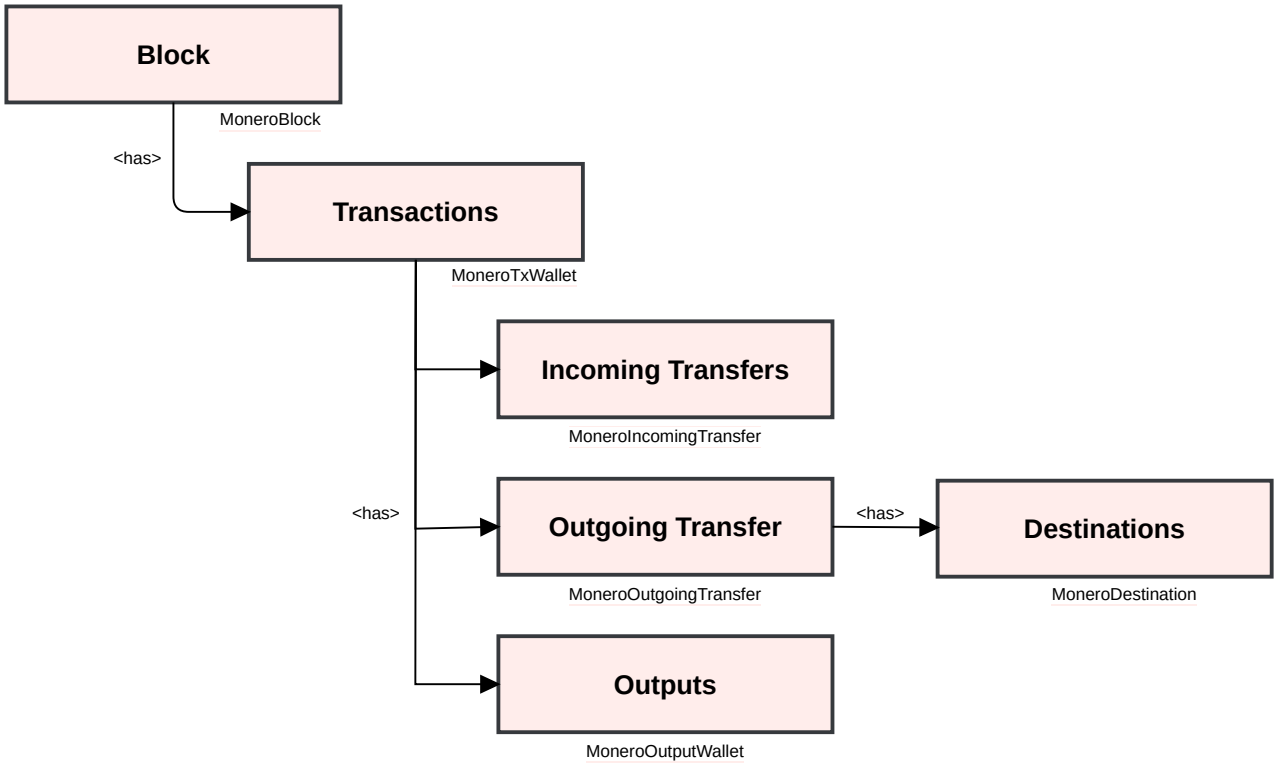
ExportMultisigHexAsync(): Task<string>

ImportMultisigHexAsync(IReadOnlyList<string> multisigHexes, bool refreshAfterImport=true): Task<int>

SignMultisigTxHexAsync(string multisigTxHex): Task<MoneroMultisigSignResult>

SubmitMultisigTxHexAsync(string signedMultisigTxHex): Task<List<string>>

Monero Wallet Types 1/4 - Transactions, Transfers, and Outputs



Transactions are part of a block and have incoming transfers and/or an outgoing transfer.

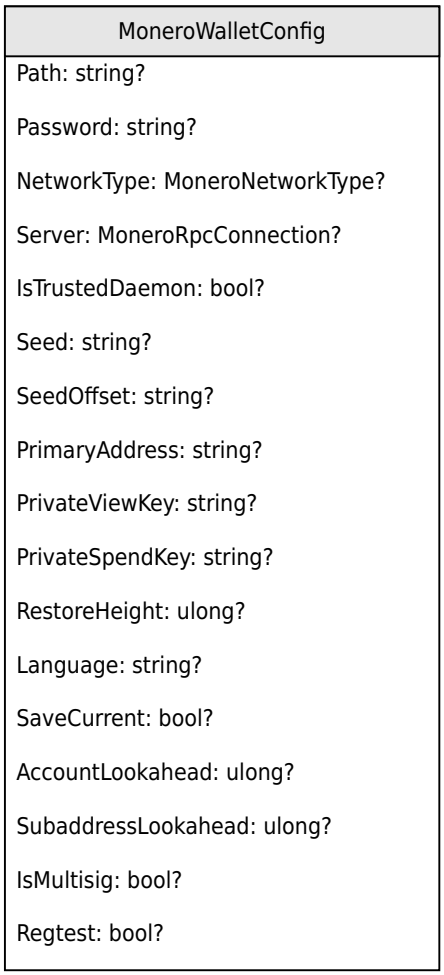
The outgoing transfer may have destination amounts and addresses which are saved locally with the wallet and are not recoverable from the blockchain.

Transactions may also have outputs received by the wallet when requested.

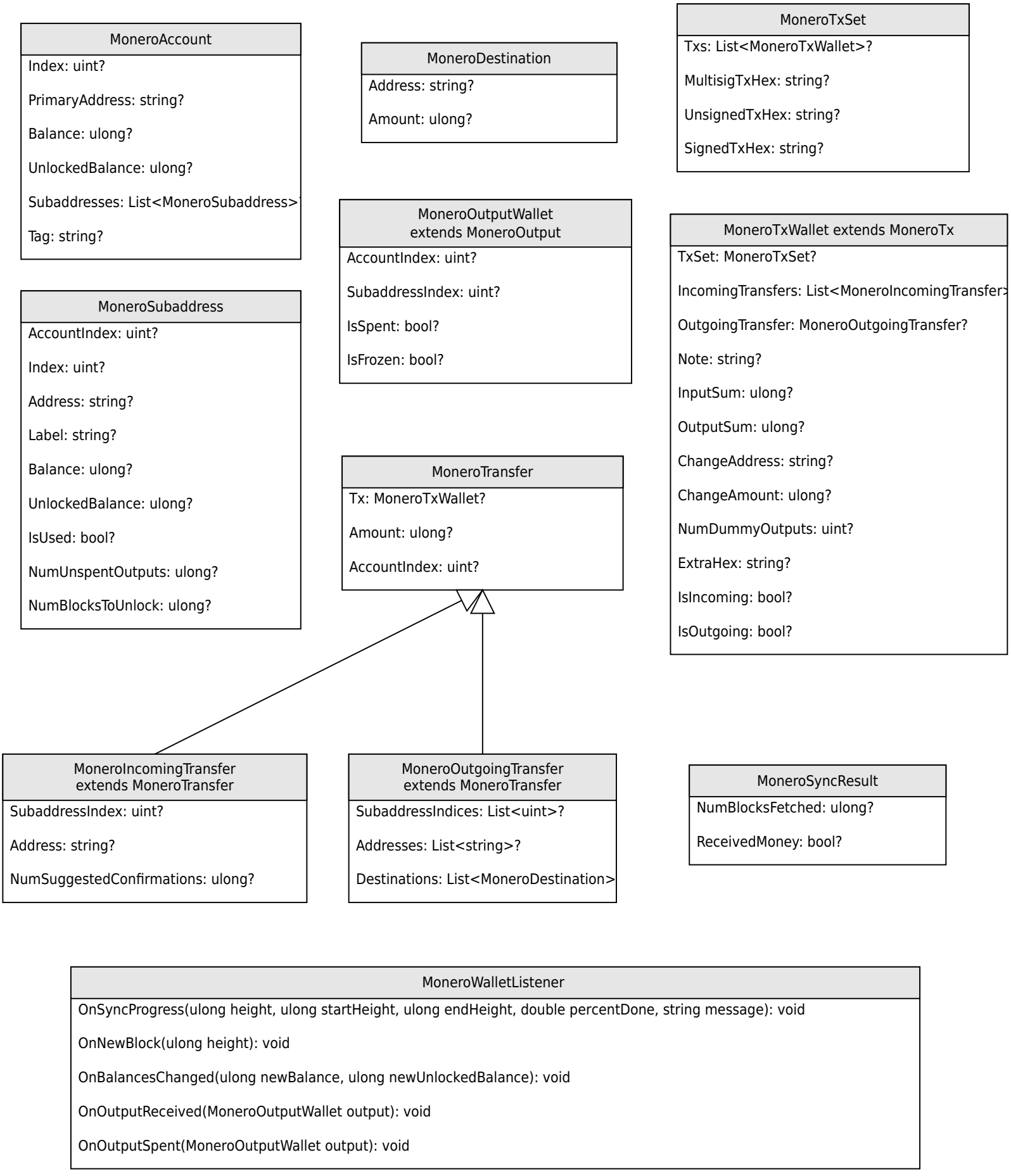
Monero Wallet Types 2/4 - Send Request and Queries for Transactions, Transfers, and Outputs



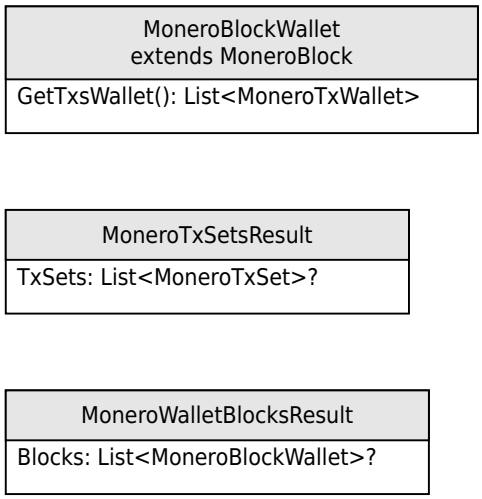
Monero Wallet Types 2/4 - Send Request and Queries for Transactions, Transfers, and Outputs (continued)



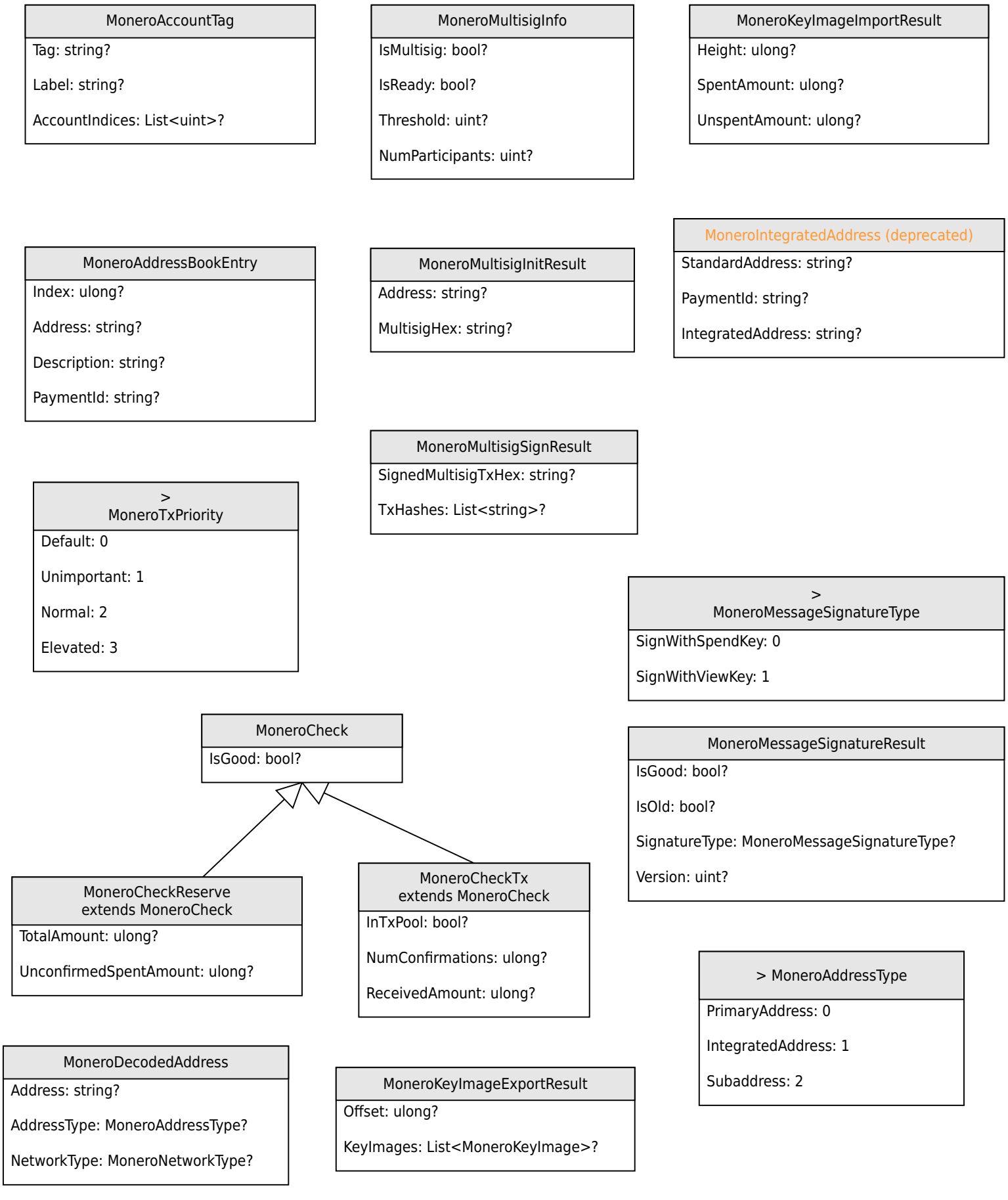
Monero Wallet Types 3/4



Monero Wallet Types 3/4 (continued)

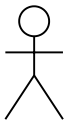


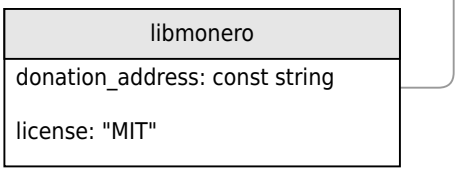
Monero Wallet Types 4/4





8BBjopSDammd96czUaBSirDgkFhoEXm3xXwqerDob1mb4kcQArRQs1fjNhPrNokJGtPkebVxX1rE5DnVF4PzSxopHZw8QpE





If this library has been valuable to you, please consider donating to support its continued development.